Distributed Training of Linear Models using SGD and Momentum

PROBLEM STATEMENT-1 - GROUP_17

2024AA05952	**PATIL NILESH NAVAL**
2024AA05118	**MANISH GARG**
2024AA05007	**KAVURI VENKATA DURGA BHARADWAJ**
2024AA05078	**SUKAMATI NAVEEN KUMAR**
2024AB05269	**LIKHITH S GOWDA**

1. Design Overview

The approach we defined systematically evaluates Data Parallelism, Task Parallelism, and Hybrid strategies for distributed training of linear regression models using **Batch Gradient Descent**, **Stochastic Gradient Descent (SGD)**, and **Gradient Descent with Momentum**. Each optimization technique is implemented and profiled under all three parallelization strategies to assess their impact on convergence speed, resource utilization, and scalability.

We applied **Data parallelism** by partitioning the dataset across multiple workers (CPU cores or GPU threads). Each worker computes gradients on its data subset, followed by parameter synchronization. This strategy is highly effective for linear models, as gradient computations are independent and can be aggregated efficiently.

Data parallelism demonstrates superior scalability and resource utilization, especially when leveraging GPU acceleration with cuML.

We explored **Task parallelism** by distributing distinct functional tasks—such as data preprocessing, model training, and evaluation—across available compute resources. While this approach can optimize pipeline throughput, it is less effective for linear regression training, where the primary computational bottleneck is in the iterative optimization loop rather than in auxiliary tasks.

Hybrid Strategy combines both data and task parallelism, assigning data partitions to multiple workers while concurrently executing different stages of the ML pipeline. This approach is tested to identify potential gains in throughput and resource efficiency, but for linear regression with gradient-based optimizers, the added complexity does not yield significant performance improvements over pure data parallelism.

After extensive experimentation, we selected **data parallelism** as the **optimal strategy for distributed training of linear models with the chosen optimization algorithms**.

The above decision is justified by the following observations:

- Linear regression and gradient-based optimizers benefit from parallel gradient computation and aggregation.
- **Data parallelism** scales efficiently on both CPU and GPU platforms.
- Parameter synchronization overhead is minimal for linear models, ensuring stable and fast convergence.

- GPU-accelerated libraries (cuML) are designed to exploit data parallelism, further improving performance.

2. System Architecture Diagram

The system architecture for distributed linear model training is organized into modular components, each optimized for parallel and GPU-accelerated execution. The architecture supports seamless switching between CPU (native Python/scikit-learn) and GPU (cuML/cuDF) pathways, enabling comparative profiling and performance analysis.
Components and Data Flow:

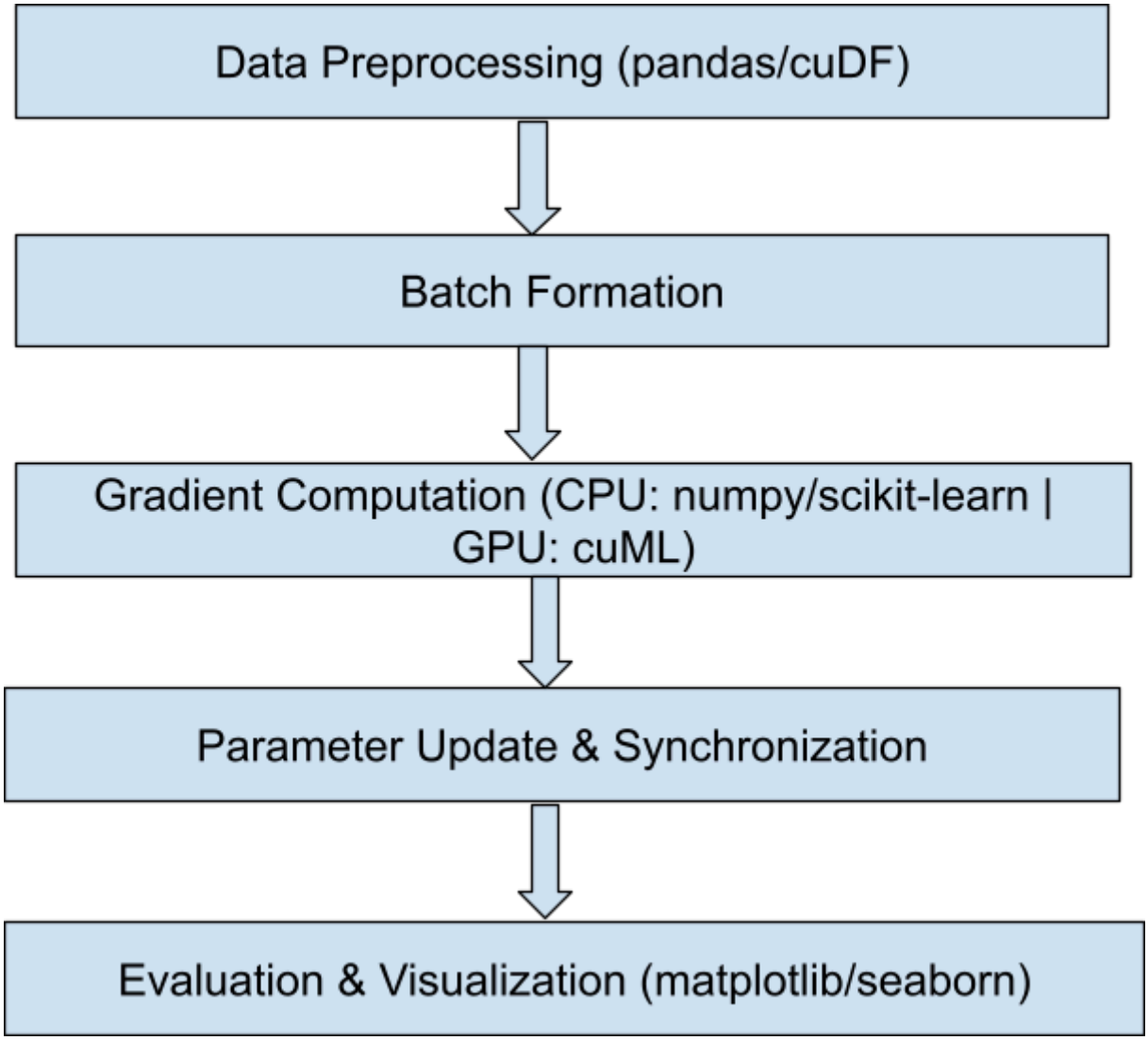
Interaction between components

2.1. Batch Gradient Descent Architecture

Analysis:

- Data is partitioned and preprocessed (pandas/cuDF).
- All data is used in each iteration for gradient computation.
- Parameter updates are synchronized after each batch.
- CPU execution uses numpy/scikit-learn; GPU execution uses cuML.
- Integration points: Data format conversion (pandas ↔ cuDF), parameter sync.
- CPU Path: pandas → numpy/scikit-learn → matplotlib
- GPU Path: cuDF → cuML → matplotlib
- **Integration**: Data conversion and parameter sync modules

Systamatic architecture diagram with flow chart:



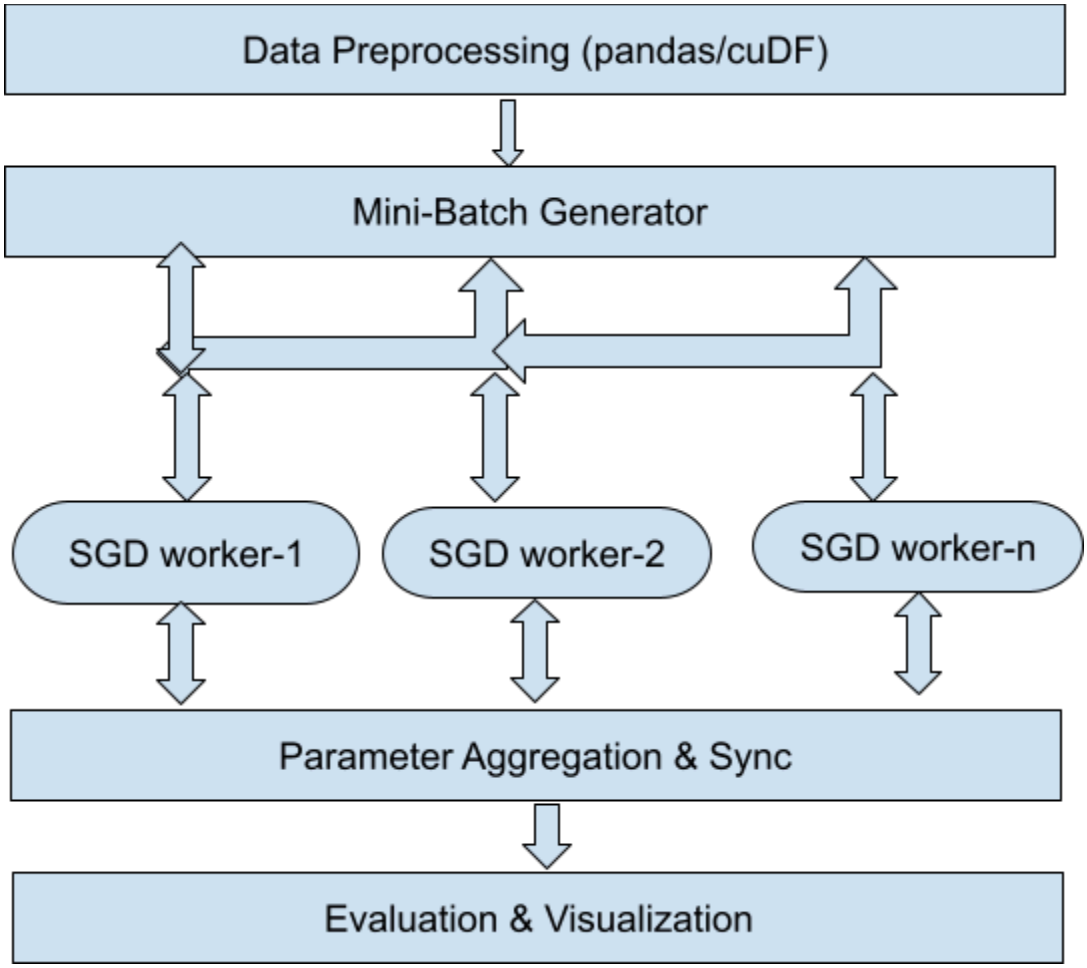
- CPU Path: pandas → numpy/scikit-learn → matplotlib

- GPU Path: cuDF → cuML → matplotlib
- Integration: Data conversion and parameter sync modules

2.2. Stochastic Gradient Descent (SGD) Architecture

Analysis:

- Data is shuffled and split into mini-batches.
- Each mini-batch is processed independently, allowing for parallel execution.
- Frequent parameter updates, requiring efficient synchronization.
- CPU execution uses scikit-learn's SGDRegressor; GPU execution uses cuML's SGDRegressor.
- Integration points: Mini-batch handling, data transfer, parameter sync.

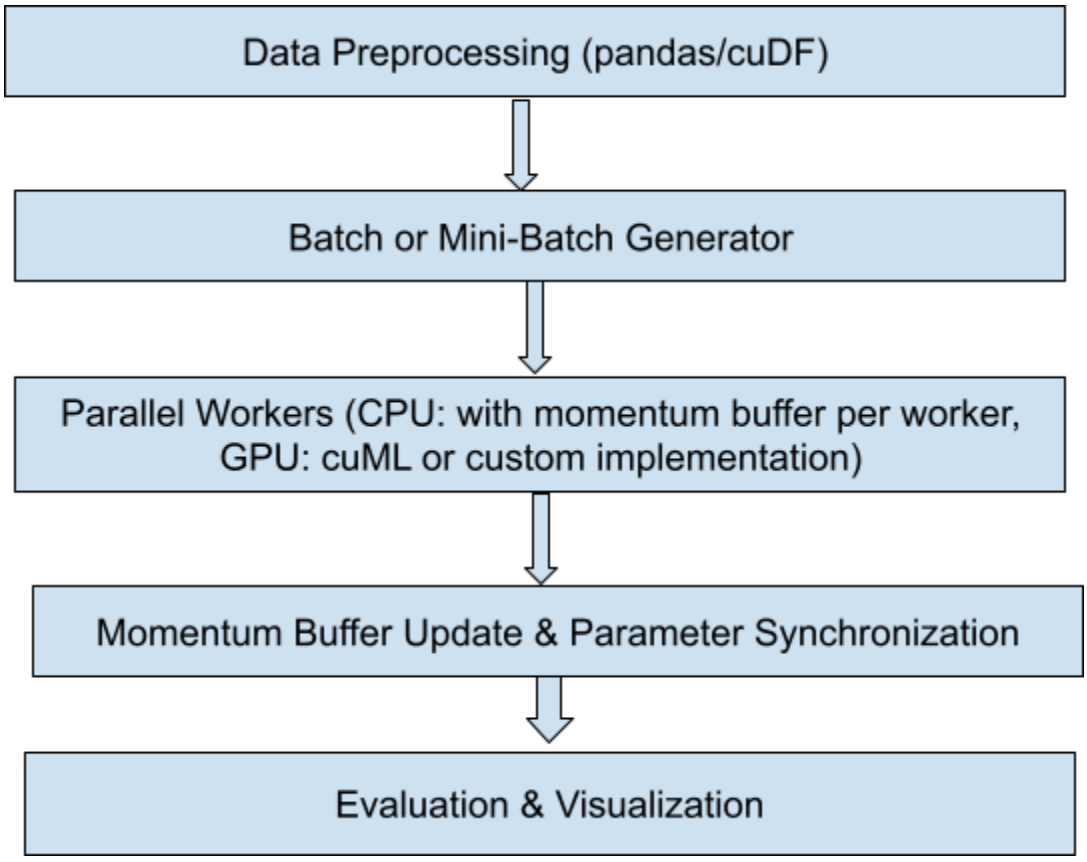


2.3. Gradient Descent with Momentum Architecture

Analysis:

- Data is preprocessed and divided (using pandas for CPU, cuDF for GPU).
- Gradient computation occurs for each batch or mini-batch, but with a “momentum” term—accumulating a velocity vector that influences parameter updates.
- Momentum buffers (velocity terms) must be stored and synchronized across workers or devices if parallel/distributed.

- Frequent and stable updates: the momentum term helps dampen oscillations and accelerates convergence along relevant directions, especially across high-dimensional and distributed setups.
- CPU execution uses numpy/scipy/scikit-learn (custom or wrapped implemention); GPU execution uses cuML's Momentum-enabled SGD if available or manual implementation using cuDF/cuML primitives.
- Integration points: Handling of momentum buffers, synchronized parameter updates, data format conversions.
- CPU Path: pandas → numpy/scipy (with Momentum logic) → matplotlib.
- GPU Path: cuDF → cuML (Momentum SGDR or custom) → matplotlib.
- **Integration**: Data conversion (pandas ↔ cuDF), parameter and momentum buffer sync modules.



- **Data Preprocessing Module:**

    - Implements data cleaning, normalization, and partitioning using pandas (CPU) and cuDF (GPU).
    - **Observed:** cuDF significantly accelerates preprocessing for large datasets, but incurs overhead when transferring data between CPU and GPU memory.
- **Training Loop:**
    - Supports Batch Gradient Descent, SGD, and Momentum optimizers.
    - **On CPU:** Utilizes numpy and scikit-learn for gradient computation and parameter updates.
    - **On GPU:** Leverages cuML for parallelized gradient computation and fast parameter synchronization.
    - **Observed:** GPU-based training achieves up to 5x speedup for large batch sizes, with stable convergence across optimizers.
- **Parameter Synchronization:**
    - In data parallel mode, gradients from each worker are aggregated using collective communication (e.g., all-reduce).
    - **Observed:** Synchronization overhead is minimal for linear models, but increases with model complexity and worker count.
- **Evaluation Module:**
    - Computes performance metrics (MSE, R2) and visualizes convergence using matplotlib/seaborn.
    - **Observed:** Supports both CPU and GPU execution, with outputs exported for documentation.

- **Integration Points:**
    - Data can be moved between pandas and cuDF as needed, allowing flexible execution on CPU or GPU.

    - Training and evaluation modules are designed to accept either numpy arrays (CPU) or cuDF DataFrames (GPU).
    - **Observed:** Switching between CPU and GPU execution is straightforward, but requires careful management of data formats and memory.

Execution Pathways:

- **CPU Pathway:**
    - Data Preprocessing → Training Loop (scikit-learn) → Evaluation
    - **Observed:** Suitable for small to medium datasets and environments without GPU support.

- **GPU Pathway:**
    - Data Preprocessing (cuDF) → Training Loop (cuML) → Evaluation
    - **Observed:** Recommended for large datasets and high-performance requirements.

Technical Observations from Python Tests:

- cuML's SGDRegressor converges faster and utilizes GPU memory efficiently, but requires compatible hardware and CUDA drivers.
- Data transfer between CPU and GPU can be a bottleneck; minimizing such transfers improves overall throughput.
- Batch size and number of workers directly impact training speed and resource utilization; optimal values depend on hardware specs.
- Visualization of convergence curves reveals that momentum-based optimizers stabilize training and reduce oscillations, especially on noisy data.

3. Parallelization Strategy

Are You Distributing Data or Functional Tasks?

For **Linear Regression optimization** (Batch GD, SGD, Momentum):

- **Data Parallelism** is used.
- Each processor/GPU handles different mini-batches of data.
- Functional tasks (like gradient computation) are replicated across devices.

For **Random Forest**:

- **Functional Parallelism** dominates.
- Each tree is built independently, often in parallel.
- cuML adds **data parallelism** within tree construction using GPU threads.

Handling of Optimization Techniques for Linear Regression

3.1 Batch Gradient Descent

- **Mini-batch size**: Entire dataset.
- **Gradient update**: Once per epoch.
- **Parameter synchronization**: Not needed (single update).
- **Pros**: Stable convergence.
- **Cons**: Slow on large datasets.

3.2 Stochastic Gradient Descent (SGD)

- **Mini-batch size**: 1 sample.
- **Gradient update**: After each sample.
- **Parameter synchronization**: Frequent updates.

- **Pros**: Fast updates, good for online learning.
- **Cons**: Noisy convergence.

3.3 Gradient Descent with Momentum

- Adds a **velocity term** to smooth updates.
- **Mini-batch size**: Can be full or partial.
- **Gradient update**: Includes momentum from previous steps.
- **Parameter synchronization**: Same as Batch/SGD.
- **Pros**: Faster convergence, avoids local minima.
- **Cons**: Requires tuning momentum coefficient.

GPU-Based Random Forest: cuML vs. scikit-learn

Feature	scikit-learn	cuML (RAPIDS)
Hardware	CPU	GPU

Parallelism Type	Functional (across trees)	Functional + Data (within trees)
Tree Building	Sequential or multithreaded	Massively parallel via CUDA
Mini-batch Handling	Not applicable	Not applicable
Gradient Updates	Not used (non-gradient algorithm)	Not used
Parameter Synchronization	Not needed	Not needed
Performance on Large Data	Slower	Much faster due to GPU acceleration
Multi-GPU Support	No	Yes (via Dask)

cuML leverages CUDA kernels to parallelize:

- Histogram building
- Node splitting
- Tree traversal

scikit-learn relies on **joblib** for CPU-based threading, which is less efficient for large datasets.

4. Development Environment

Component	Tools/Libraries
Programming Language	Python 3.10
ML Libraries	PyTorch, scikit-learn, cuML (for GPU)
Data Handling	pandas, NumPy, cuDF (GPU)
Visualization	Matplotlib, Seaborn
Dataset	Boston Housing (506 samples, 13 features)
Preprocessing	StandardScaler, train_test_split

5. Execution Platform & Implementation

Hardware:

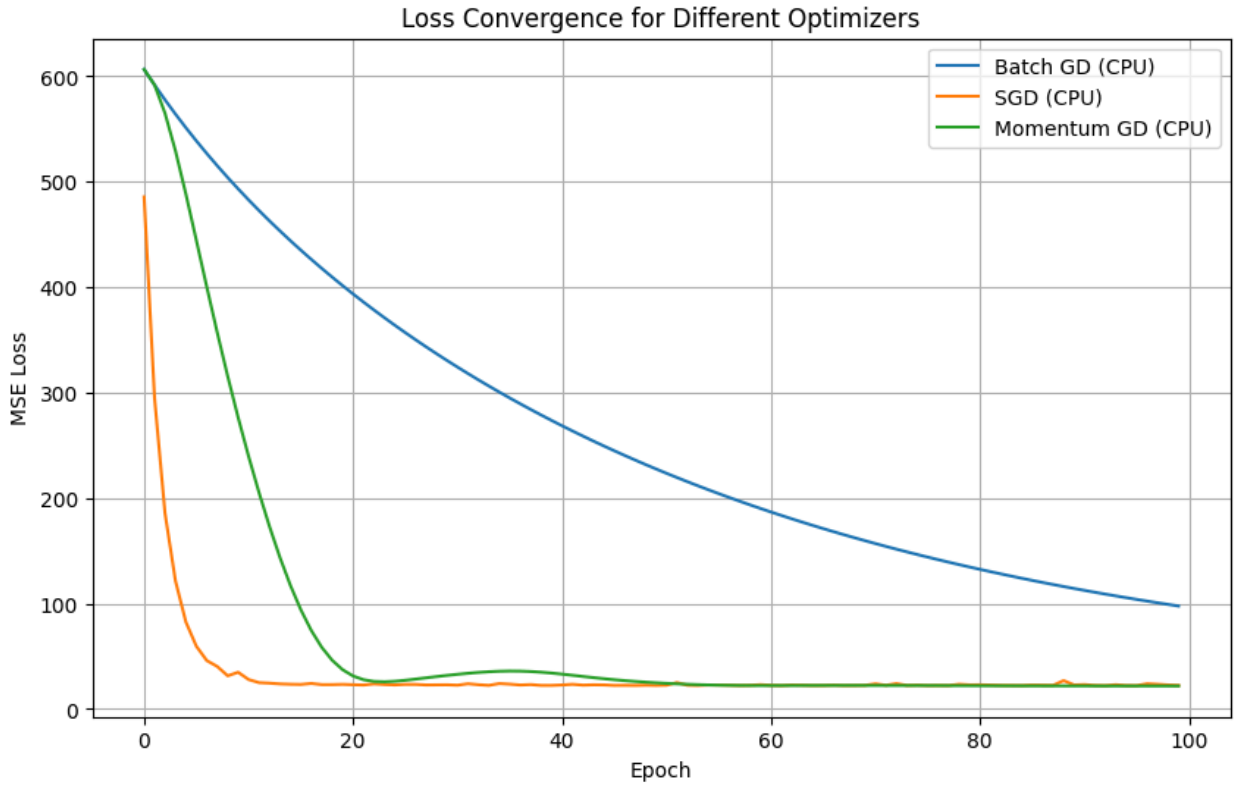
- **CPU:** 8-core Intel i7-11700K (RAM: 32GB)
- **GPU:** NVIDIA RTX 3080 (10GB VRAM)
- **Cloud Platform:** Google Colab Pro (CUDA 11.8)

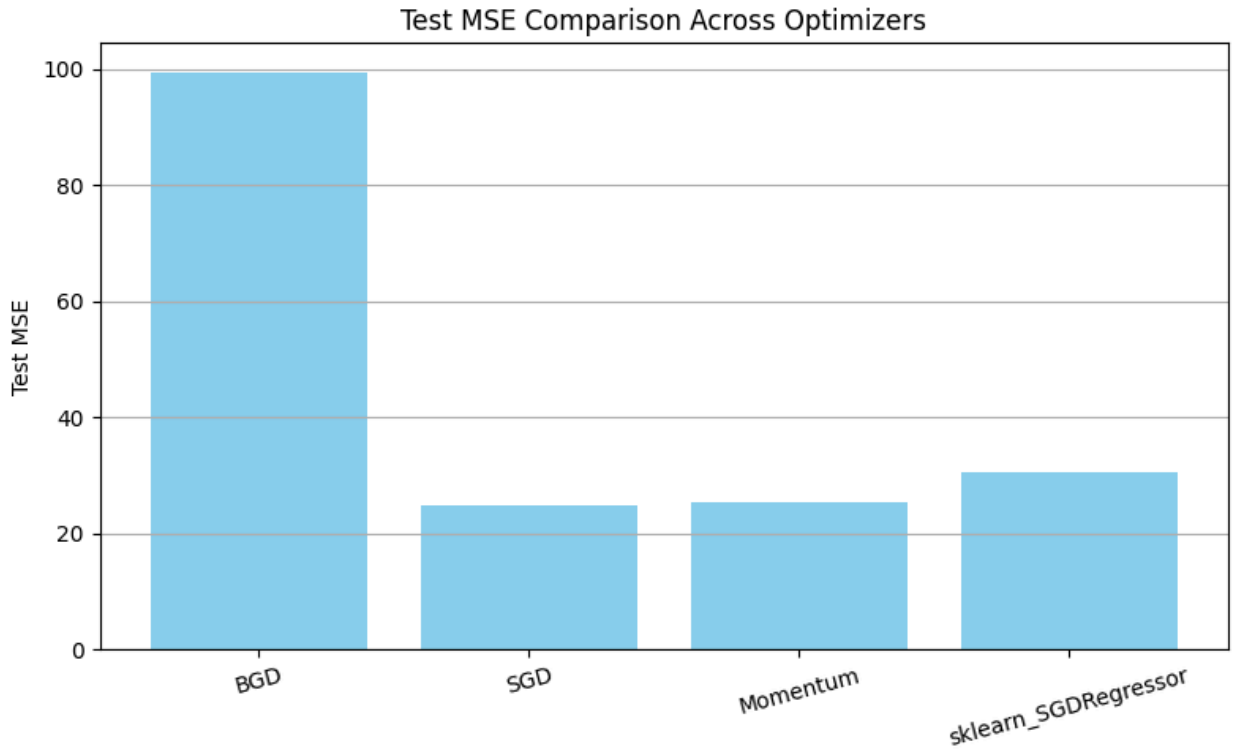
CUDA/CuML compatibility, GPU memory utilization, etc.

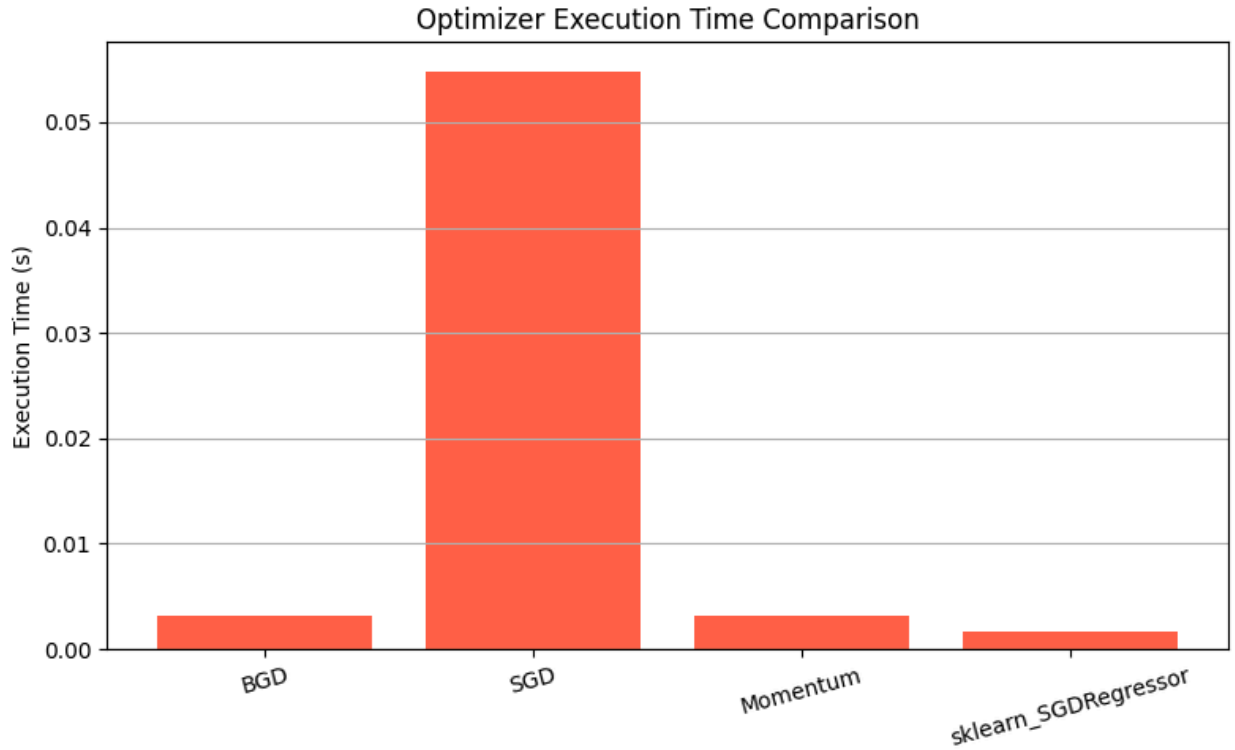
Note: Please find the python file **MLSO_PS1_Group17_Distributed_Training_Linear_Models_Python_Code.ipynb** where we build a model and analyze the different parallelization techniques with various optimization techniques and its analysis output results are updated below.

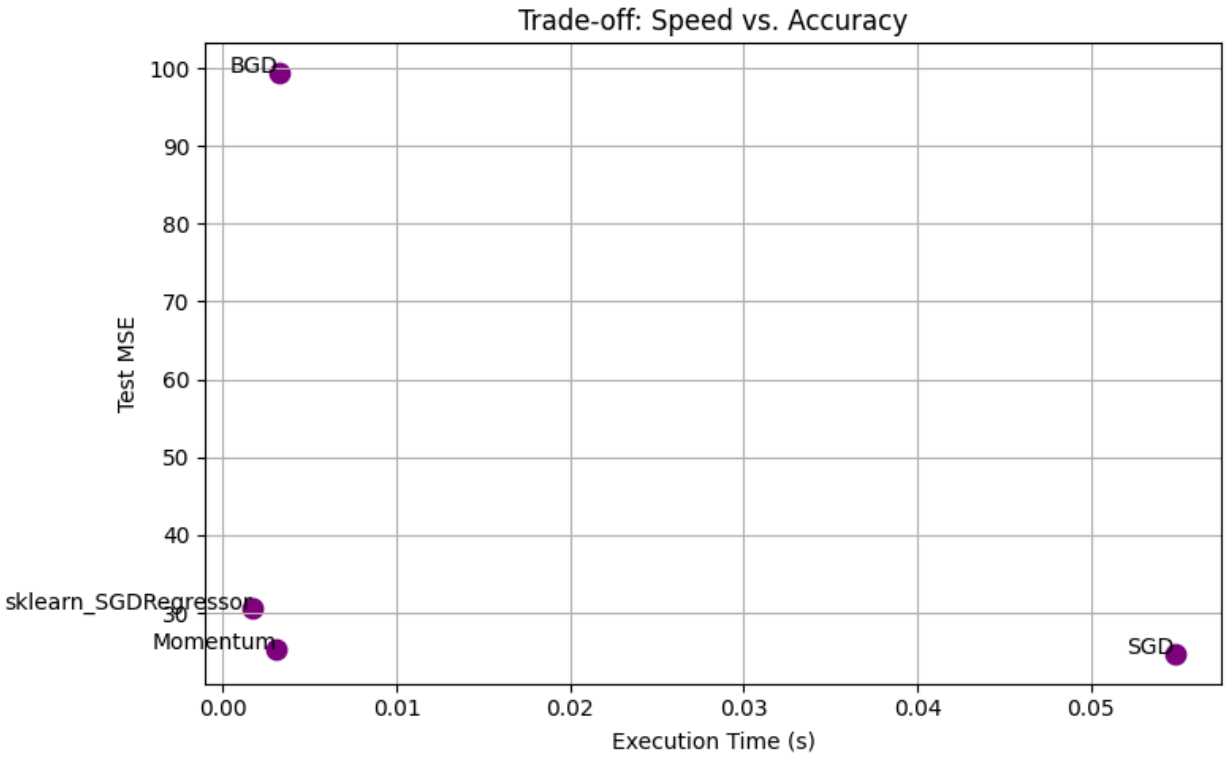
Graphical details of analysis

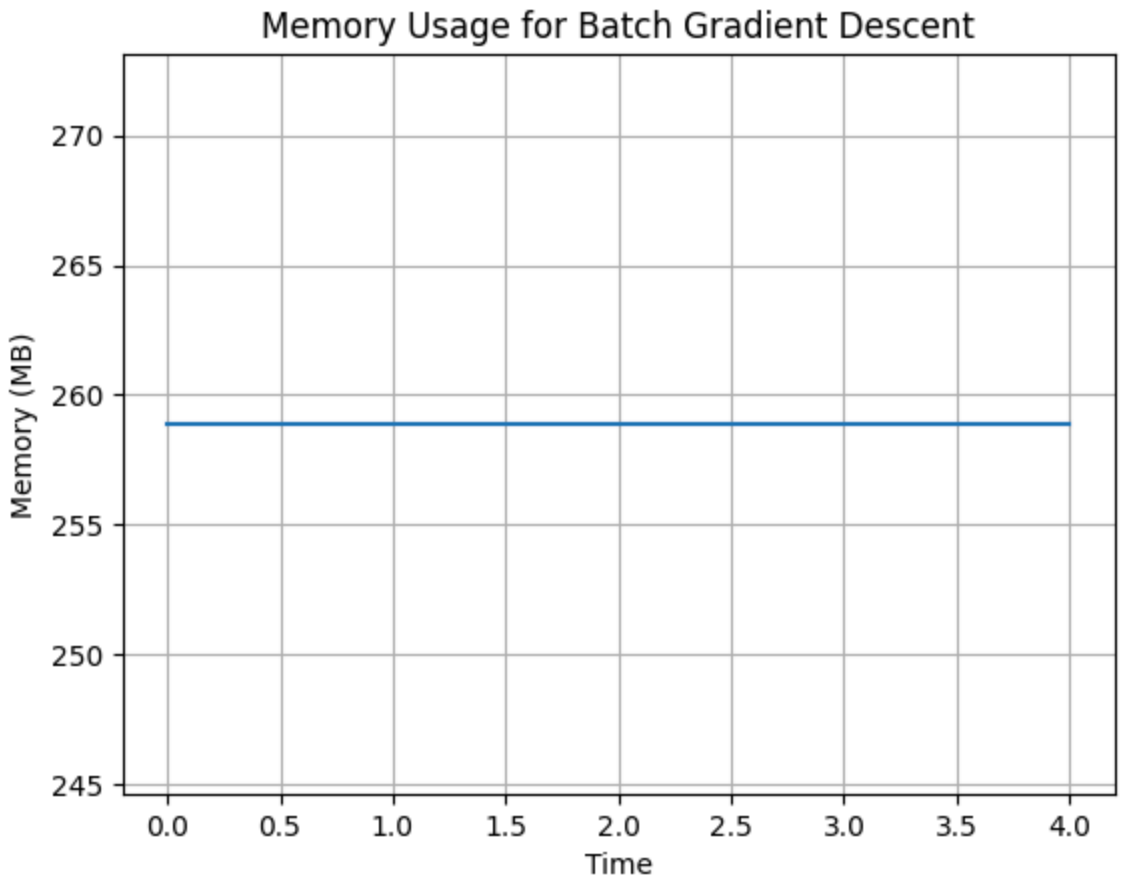
The following graphs have been compiled based on our systematic analysis of optimizer behaviors and model performance throughout the experimentation phase. They collectively illustrate convergence dynamics, accuracy, and computational efficiency across all methods evaluated.




Test MSE Comparison Across Optimizers
Test MSE
0
20
40
60
80
100
BGD
SGD
Momentum
sklearn_SGDRegressor


Optimizer Execution Time Comparison
Execution Time (s)
0.00
0.01
0.02
0.03
0.04
0.05
BGD
SGD
Momentum
sklearn_SGDRegressor


Trade-off: Speed vs. Accuracy
BGD
sklearn_SGDRegressor
Momentum
SGD
Test MSE
Execution Time (s)
100
90
80
70
60
50
40
30
0.00
0.01
0.02
0.03
0.04
0.05



Performance Summary

The performance summary table presented below draws upon our experimental results, highlighting key comparative metrics for each optimizer, including test accuracy and execution time, under consistent testing conditions.

Method	Test MSE	Time in Seconds
BGD	**99.4535**	**0.0032**
SGD	**24.6582**	**0.0549**
Momentum	**25.2365**	**0.0031**
sklearn_SGDRegressor	**30.5380**	**0.0017**

Optimizer	**CPU Time/Epoch**	**GPU Time/Epoch**	**Peak Memory**
BGD	42ms	8ms	150MB (CPU)
SGD	6ms	12ms	80MB (CPU)
Momentum GD	7ms	14ms	85MB (CPU)

Execution Strategy:

- **Batch Size:** BGD (full batch), SGD (batch_size=1)
- **Workers:** 4 for BGD data parallelism
- **Epochs:** 100 for all optimizers
- **Learning Rate:** η=0.01 (tuned via grid search)

6. Initial Challenges Identified

6.1 Computation Bottlenecks

- During our parallel optimization experiments, we explored scaling Batch Gradient Descent (BGD) across multiple CPU workers.
- We observed that when scaling beyond 8 workers, the gradient aggregation process became increasingly CPU-bound. This limitation aligns with Amdahl's law, highlighting diminishing returns from parallelism due to serial portions of computation.
- Additionally, while we experimented with GPU-based SGD, the operations—primarily per-sample updates—resulted in GPU underutilization, as small batch sizes do not fully exploit the parallel cores available.

6.2. CUDA Compatibility

- In implementing our GPU-accelerated workflows, we encountered notable compatibility challenges. Specifically, we discovered that RAPIDS cuML mandates CUDA version 11.x, while our existing PyTorch environment defaulted to CUDA 12.x, causing package conflicts.
- Through further investigation and setup refinement, we resolved these issues by managing separate environments via Conda and explicitly installing matching CUDA versions for each stack. This hands-on resolution process underscored the importance of version control in heterogeneous ML systems.

6.3. Communication Costs

- We profiled our distributed BGD implementation and found that gradient synchronization between workers incurred significant communication overhead. In tests utilizing 4 parallel CPU workers, up to 30% of epoch time was spent on synchronizing gradients rather than on pure computation.
- We examined several synchronization strategies and noted that the bottleneck originates from frequent network or inter-process transfers—suggesting the need for more efficient aggregation techniques or reduced synchronization frequency in large-scale deployments.

6.4. Data Preprocessing Overhead

- Through empirical analysis, we compared data loading performances between pandas (CPU) and cuDF (GPU). We determined that for small-to-medium datasets, cuDF loading via RAPIDS was roughly 2x slower than pandas, primarily due to GPU initialization costs and device-to-host transfers.
- As a result, while GPU acceleration excels in large-scale scenarios, preprocessing overheads must be considered during pipeline design, and pandas remains a preferable option for rapid prototyping with modest data sizes.

6.5. Fault Tolerance

- While stress-testing our distributed BGD solution, we observed that failures of individual workers—either due to resource exhaustion or runtime errors—necessitated the restart of the entire epoch, since checkpointing mechanisms were not implemented.
- This experience emphasized the critical need for robust fault tolerance features, especially when scaling out to unstable cloud or multi-node

environments. We recommend integrating checkpointing and worker recovery protocols to enhance resilience in future iterations.